Modélisation VHDL et implémentation d'un processeur RISC 16 bits

Objectif :

Nous allons modéliser un processeur RISC 16 bits. Pour ce faire, nous allons le décrire intégralement en VHDL, puis nous l'implanterons sur un FPGA (*Field-Programmable Gate Array*), un circuit intégré complètement programmable capable d'implémenter n'importe quel circuit modélisé en HDL.

1. Processeur RISC 16 bits

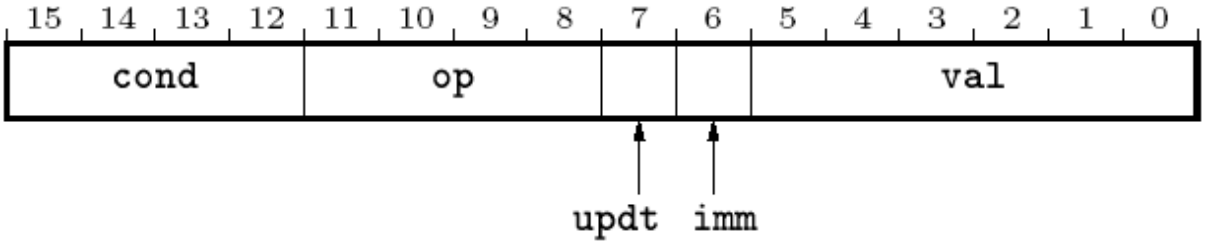
Le processeur considéré est à une adresse et compte 64 registres de 16 bits. Il dispose donc d'un accumulateur Acc correspondant au registre R0, ainsi que d'un compteur ordinal (PC, pour *program counter*) correspondant à R63. On a aussi un compteur ordinal de retour, dans lequel est stocké l'adresse de retour après un appel de fonction ; il est noté RPC et fixé à R62.

Nom du registre	numéro
accumulateur	R0
PC	R63
RPC	R62

1.1 Mot d'instruction

Le mot d'instruction est découpé de la manière suivante :

- bits 15 à 12, `cond` : prédicat conditionnant l'exécution de l'instruction en fonction des bits de statut (Tableau 2)
- bits 11 à 8, `op` : opcode de l'instruction ;
- bit 7, `updt` : drapeau déterminant si les bits de statut doivent être mis à jour par cette instruction;
- bit 6, `imm` : drapeau déterminant si le champ suivant correspond à un numéro de registre ou à un immédiat ;
- bits 5 à 0, `val` : numéro de registre rX si `imm` est à 0, ou valeur immédiate X si `imm` est à 1.



1.2 Jeu d'instructions

Le jeu d'instructions est très simple, puisqu'il n'y a que 16 instructions possibles décrites dans le tableau-ci-dessous.

Tableau 1 : Jeux d'instructions

Opcode	Codage	Action
AND	0000	Acc ← Acc AND rX/X
OR	0001	Acc ← Acc OR rX/X
XOR	0010	Acc ← Acc XOR rX/X
NOT	0011	Acc ← NOT rX/X
ADD	0100	Acc ← Acc + rX/X
SUB	0101	Acc ← Acc − rX/X
LSL	0110	Acc ← Acc ≪ rX/X
LSR	0111	Acc ← Acc ≫ rX/X

Opcode	Codage	Action
LDA	1000	Acc ← [rX/X]
STA	1001	[rX/X] ← Acc
MTA	1010	Acc ← rX/X
MTR	1011	rX ← Acc
JRP	1100	PC ← PC + rX/X
JRN	1101	PC ← PC − rX/X
JPR	1110	PC ← rX/X
CAL	1111	RPC ← PC + 1 PC ← rX/X

À noter :
- MTA, *Move To Accumulator* ;
- MTR, *Move To Register* : le bit imm ne doit pas être à 1 (ceci est en théorie assuré par le compilateur assembleur) ;
- JRP, *Jump Relative Positive* ;
- JRN, *Jump Relative Negative* ;
- JPR, *JumP Register* ;
- CAL, *CALl* : sauvegarde la valeur incrémentée du PC dans RPC (*return program counter*, fixé à r62), puis effectue un JPR. Pratique pour un appel de fonction.

Syntaxe:

- **<mnémonique>{<cond>}{S} Operand**

Exemple:

- **Instruction arithmétique et logique :**

  a. adressage registre :

  ANDT r1 (AND True) → Toujours accumulateur = accumulateur and r1 et ne pas mettre à jours les bits de flag

  b. adressage immédiat :

  SUBF 0x23 (SUB False) : → Jamais accumulateur = accumulateur - 0x23, cette instruction ne sera jamais exécuter par le processeur après décodage.

- **Instruction de transfert :**

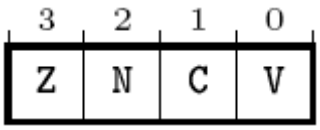
  MTAP r3 (Move to accumulateur if Positif) → si Z=0 et N=0 alors accumulateur = r3
  MTAN 0xAC (Move to accumulateur if Negatif) → si N=1 alors accumulateur = 0X00AC

1.3 Statut et conditions

On se donne les 4 drapeaux de statut suivants :

- **Z, *Zero*** : 1 si le résultat d'une opération est nul, et 0 sinon ;
- **N, *Negative*** : 1 si le résultat est strictement négatif, et 0 sinon ;

- **C, *Carry*** : 1 s'il y a dépassement de capacité sur 16 bits, et 0 sinon. Pour les instructions de décalage vers la gauche ou vers la droite le bit carry prend 1 si au moins un bit éjecté de la donnée décalée est égal à 1.
- **V, *oVerflow*** : 1 s'il y a dépassement de capacité sur 15 bits (dépassement signé), et 0 sinon.
  - Pour l'addition : il y a un overflow si on additionne 2 nombres positifs et le résultat est négatif ou si on additionne 2 nombres négatifs et on obtient un résultat négatif.
  - Pour la soustraction A- B : il y a un overflow lorsque les 2 opérandes sont de signes opposés et que le signe du résultat est différent du signe de A.
  - Pour les instructions de décalage vers la gauche : il y a un overflow lorsqu'on décale un nombre Positif et le résultat est négatif ou l'inverse.



- Les instructions logiques ne mettent à jours que les bits N et Z
- Pour les instructions de décalage, le bit de flag C est utilisé pour stocker le dernier bit décalé ; il y a un overflow pour les instructions de décalage si l'opérande après décalage change de signe

1.4 Conditions

L'exécution de toutes instructions est conditionnelle, en effet une instruction n'est exécutée que si la condition est vérifiée. En général une condition est rajoutée au mnémonique de l'instruction.

Les prédicats conditionnant l'exécution des instructions sont alors codés de la manière suivante :

Tableau 2 : **Conditions d'exécution**

Condition	Codage	Expression
F	0000	0
T	0001	1
Z	0010	Z
NZ	0011	$\neg$Z
P	0100	$(\neg Z) \wedge (\neg N)$
NP	0101	$Z \vee N$
N	0110	N
NN	0111	$\neg$N

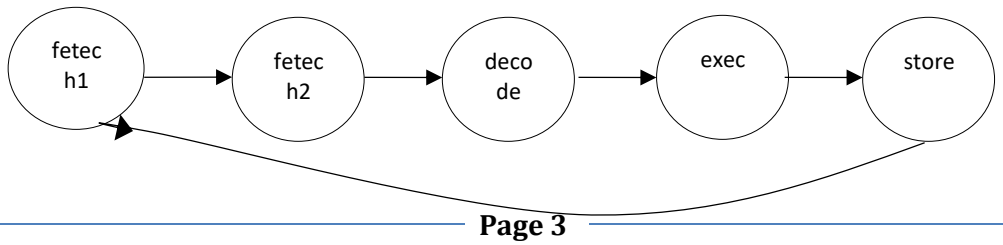
Condition	Codage	Expression
C	1000	C
NC	1001	$\neg$C
V	1010	V
NV	1011	$\neg$V
–	1100	–
–	1101	–
–	1110	–
–	1111	–

Exemple

- si la condition est 0010, l'instruction ADDZ ne s'exécute que si le bit Z = status(3)=1
- si la condition est 0100, l'instruction ANDP ne s'exécute que si le bit Z=0 **et** le bit N=0
- si la condition est 0001, l'instruction MTAT s'exécute toujours

1.5 Cycle d'exécution

Toutes les instructions prennent 5 cycles processeur pour s'exécuter selon la FSM suivante :



- **fetch1**: la valeur de PC est mise sur le bus d'adresse vers la mémoire.
- **fetch2:** le mot d'instruction lu en mémoire est stocké dans le registre d'instruction.
- **decode:** les valeurs des opérandes de l'instruction sont stockées dans les registres op1 et op2.
- **exec:**
  - L'UAL calcule le résultat de l'opération ;
  - L'accès mémoire (en lecture ou en écriture) est aussi initié ;
  - Le PC est incrémenté puis stocké dans PC (toute les instructions) ou dans RPC (si CAL)
- **store**: le résultat de l'ALU ou de la lecture en mémoire est mémorisé dans le registre adéquat. Pour l'instruction CAL, écriture dans PC

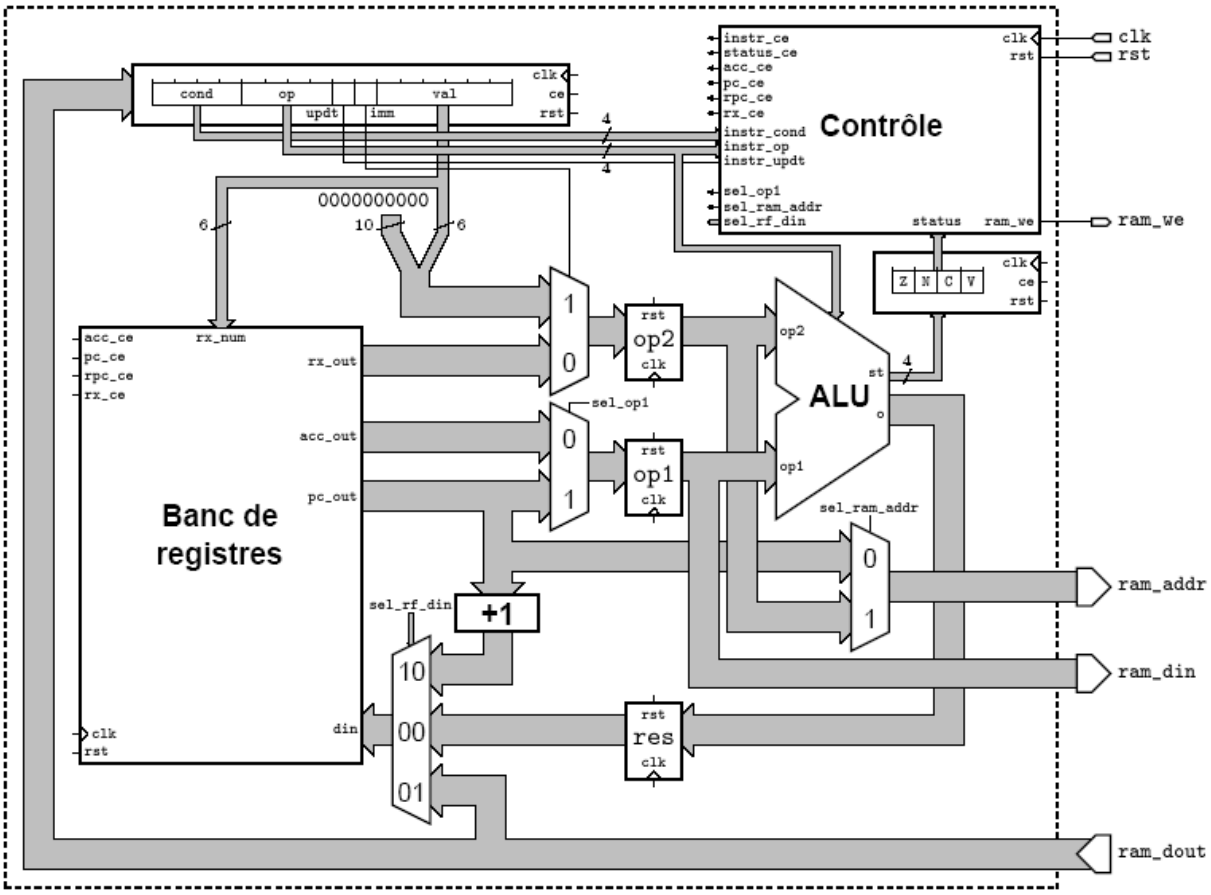


FIG. 1 – Architecture générale du processeur.

2. Réalisation

Pour réaliser ce processeur, vous allez utiliser Modelsim.

1.Créer un projet nommé processeur et y rajouter des fichiers.vhd pour décrire les différents composants de ce processeur. Commencer par décrire :

- les registres (instruction, statut et banc de registres),

- l'ALU,
- l'unité de contrôle,
- l'architecture globale du processeur sur laquelle viendront se greffer les composants précédents.

Un dossier vous est fournit servant d'ébauche de modèles VHDL ainsi que des test benchs pour valider vos composants. Il faut donc déterminer la description comportementale au niveau RTL (Register Transfer Level) écrite en langage VHDL de votre composant. *Simuler* et vérifier son bon fonctionnement en utilisant l'outil **`ModelSim`**.

2.1 Registre de statut

Même topologie que pour le registre d'instruction, mis à part que celui-ci ne fait que 4 bits. Les drapeaux de statut arrivent de l'ALU par le port i, puis sont stockées dans le registre lorsque le signal de *clock enable* **`ce`** est à 1.

2.2 Banc de registres

Le banc de registre contient donc 64 registres de 16 bits. Tous ces registres sont accessibles grâce au port rx_num. Ainsi, le contenu du registre sélectionné par rx_num se retrouve sur rx_out grâce à un multiplexeur.

L'accumulateur étant r0 et le PC étant r63, ces deux registres proposent aussi directement leur contenu en sortie sur les ports acc_out et pc_out respectivement --> **la lecture est alors asynchrone**

Enfin, l'écriture dans le banc de registre s'effectue par le port din. Le registre dans lequel cette valeur sera stockée est indiqué par les différents signaux de *clock enable* :

- acc_e s'il s'agit de l'accumulateur,
- pc_ce s'il s'agit du PC,
- rpc_ce s'il s'agit du RPC (r62),
- et rx_ce s'il s'agit du registre sélectionné par rx_num (nécessite donc un démultiplexeur).

Remarque : Lors d'un reset, le PC doit être réinitialisé à 0x00,

2.3 Unité arithmétique et logique

Vous pouvez ici aussi vous répartir les tâches entre les différentes opérations si vous le souhaitez, mais une approche unifiée pour minimiser la logique serait préférable.

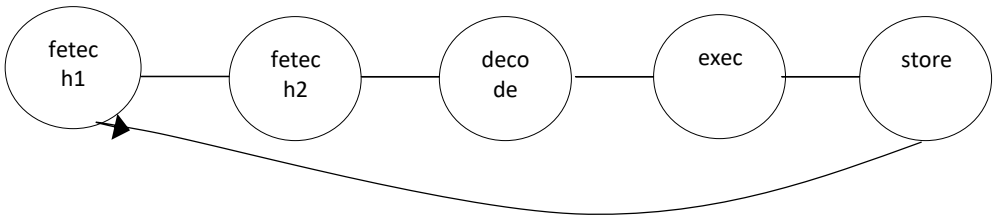
Il faut donc que votre ALU réalise les opérations suivantes :

- AND, OR, XOR et NOT bit à bit : vous pouvez utiliser pour cela les versions vectorielles des instructions and, or, xor et not.
- Addition et soustraction : de même, pour vous simplifier la vie, utilisez les opérateurs + et -. Attention aux dépassements de capacité !
- Décalages arithmétiques à gauche et à droite :
- Encore une fois, attention aux dépassements de capacité !
- Recopies (pour les instructions comme MTA ou MTR) : notez que les déplacements de registre à registre passent par l'ALU pour pouvoir mettre les drapeaux de statut à jour.

Pensez aussi à mettre correctement à jour les drapeaux de statut en fonction de la valeur du résultat.

2.4 Unité de contrôle

L'unité de contrôle est composée d'un automate FSM qui enchaîne de manière cyclique les états fetch1, fetch2, decode, exec et store.



En fonction de l'état courant stocké dans le registre state_r ainsi que d'autres paramètres

tels que l'instruction ou le statut, cette unité doit générer les signaux de commande appropriés pour contrôler les écritures dans les registres (*_ce) ou en mémoire (ram_we) ainsi que les multiplexeurs (sel_*). Ces signaux de commandes sont:

- instr_ce,
- acc_ce,
- pc_ce,
- rpc_ce ,
- rx_ce ,
- ram_we ,
- sel_ram_addr,
- sel_op1,
- sel_rf_din

1. Décrire tout d'abord la machine d'états (FSM) de l'unité de commande en précisant pour chaque état les valeurs des sorties (la µinstruction à envoyer au chemin de données)

2. Procéder à sa description en VHDL niveau RTL (voir le chapitre 2).

2.5 Architecture globale du processeur

Il s'agit ici de relier tous les composants (registres, ALU, contrôle) entre eux, en insérant multiplexeurs et registres correctement contrôlés aux endroits nécessaires. Il suffit en gros de suivre le schéma global du processeur donné précédemment.

un fichier .vhd sert alors de décrire l'architecture globale d'une façon structurelle en instanciant tous les composants déjà décrit.

2.6 Synthèse et implémentation sur FPGA :

Après validation du processeur entier avec toutes ses unités, procédez à la synthèse du code VHDL, le placement puis le routage en ciblant le FPGA Spartan 3^E comme matériel d'implémentation.

- Etudiez la fréquence maximale de fonctionnement de notre processeur pour la cible hardware choisie
- Etudiez le nombre de porte logique, de Slices, de CLB nécessaire à l'implémentation de ce processeur.